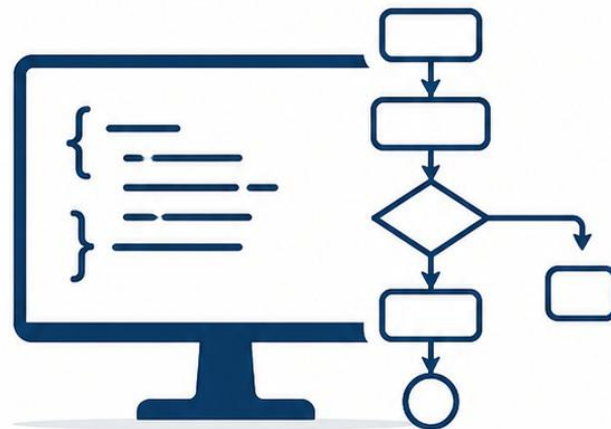


ALGORITMIA

DISEÑO • ANÁLISIS • EFICIENCIA



Alumno:

Noel Suárez Menéndez



Grupo de laboratorio:

L02



Asignatura:

Algoritmia

ÍNDICE

Práctica 0	-----	3
Práctica 1.1	-----	5
Práctica 1.2	-----	10
Práctica 2	-----	12
Práctica 3	-----	16
Práctica 4	-----	22
Práctica 5	-----	23
Práctica 6	-----	24
Práctica 6E	-----	28
Práctica 7	-----	29

P0

SE LE PIDE: Haga una tabla en la que refleje los tiempos de ejecución del módulo PythonA1.py para los valores de n expuestos (5000, 10000, 20000, 40000, 80000, ...). Si para alguna n tardara más de 60 segundos puede poner FdT (“Fuera de Tiempo”), tanto en este apartado como en los apartados posteriores.

Tiempo del PythonA1 en clase (Intel Core i5 12th @2,50 GHz)					
n	5000	10000	20000	40000	80000
t (ms)	333	1339	5441	22332	88697
					FDT

SE LE PIDE: Haga una tabla en la que refleje, al menos para dos ordenadores a los que tenga acceso, los tiempos de ejecución del módulo PythonA1.py para los valores de n expuestos (5000, 10000, 20000, 40000, 80000, ...). Referencie claramente para cada ordenador qué CPU es la existente.

Tiempo del PythonA1 en clase (Intel Core i5 12th @2,50 GHz)					
n	5000	10000	20000	40000	80000
t (ms)	333	1339	5441	22332	88697
					FDT

Tiempo del PythonA1 en casa (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
t (ms)	374	1537	6375	67478	135137
				FDT	FDT

SE LE PIDE: Ejecutar la clase que se le proporciona de nombre JavaA1.java, que utiliza el mismo algoritmo A1 para ver resolver el *PROBLEMA EJEMPLO*.

Hacer una tabla en la que refleje los tiempos de ejecución de JavaA1.java para los valores de n expuestos (5000, 10000, 20000, 40000, 80000, ...).

Los tiempos se han de obtener de la doble forma: *JAVA SIN OPTIMIZACIÓN* y *JAVA CON OPTIMIZACIÓN*.

Para finalizar, compare esos tiempos entre sí y con los obtenidos en Python (en un apartado anterior), para ese mismo algoritmo A1.

Tiempo del PythonA1 en casa (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
t (ms)	374	1537	6375	67478	135137
				FDT	FDT

Tiempo del JavaA1 (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
t (ms)	104	403	1577	14241	91340
					FDT

Tiempo del JavaA1 optimizado (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
t (ms)	27	74	300	1210	8639

SE LE PIDE: Tras estudiar PythonA2.py, PythonA3.py y PythonA4, ejecútelas y ponga en una tabla sus tiempos de ejecución, para los valores de n ya antes expuestos (5000, 10000, 20000, 40000, 80000...).

Realice la codificación de esos mismos algoritmos A2, A3 y A4 en Java, en tres clases de nombres respectivamente JavaA2.java, JavaA3.java y Java A4.java. 5

Posteriormente, realice una tabla en la que refleje los tiempos de ejecución (de ambas formas: *JAVA SIN OPTIMIZACIÓN* y *JAVA CON OPTIMIZACIÓN*) de las clases JavaA2.java, JavaA3.java y JavaA4.java, para los valores de n expuestos (5000, 10000, 20000, 40000, 80000, ...).

Para finalizar, compare esos tiempos con los antes obtenidos en el entorno Python para esos mismos algoritmos.

Tiempos de los PythonA2-4 (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
PythonA2 t (ms)	52	177	666	2665	12175
PythonA3 t (ms)	42	87	304	1241	4608
PythonA4 t (ms)	1	3	5	9	14

Tiempos de los JavaA2-4 Sin Optimizar (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
JavaA2 t (ms)	18	42	166	602	2383
JavaA3 t (ms)	17	25	82	304	1233
JavaA4 t (ms)	4	0	0	0	2

Tiempos de los JavaA2-4 Optimizados (Intel Core i7 12th @1,7 GHz)					
n	5000	10000	20000	40000	80000
JavaA2 t (ms)	13	9	31	110	438
JavaA3 t (ms)	8	4	15	62	215
JavaA4 t (ms)	6	0	0	1	0

P11

SE LE PIDE: ¿calcular cuántos años más podremos seguir utilizando esta forma de contar? Explica el razonamiento seguido para realizar el cálculo.

Un long son 64 bits, lo que da para 18.446.744.073.709.551.616 combinaciones, pero al ser con signo el valor máximo es 9.223.372.036.854.775.807 y desde que se empezó a usar, se va por el numero 1.770.228.540.138, por lo que el numero de años que quedan es el siguiente: $2,9 \cdot 10^8 - 56$ años

Tabla para las ejecuciones usando –Xint Vector2

n	1.000.000	10.000.000	20.000.000	50.000.000	100.000.000	200.000.000
T (ms)	5	50	95	239	477	977

SE LE PIDE: ¿Por qué a veces el tiempo medido sale 0? ¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?

Porque a veces actúa el recolector de basura, modificando los tiempos o porque el algoritmo se ejecuta en menos de un 1 ms y la diferencia entre el t inicial y el t final puede ser 0 aunque el t real no lo sea. A partir de tamaños suficientemente grandes donde el tiempo sea medible con el formato usado, como un $n > 30000$, aproximadamente.

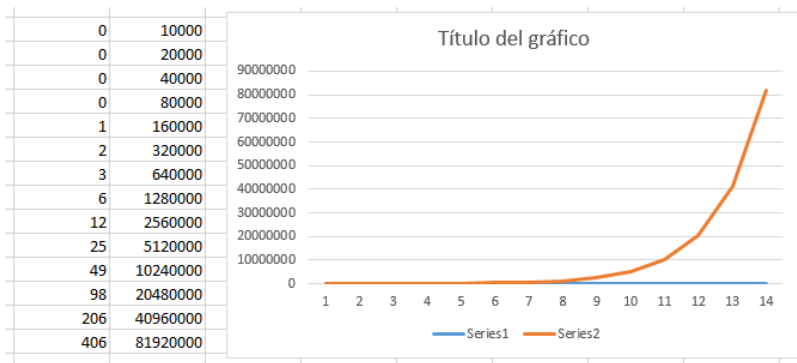
Ejecuciones usando –Xint Vector3

```
C:\Users\uo307928\Desktop\alg_NoelSuarezMenendezU0307928\p11>java -Xint p11.Vector3 100000000
n  tTiempo
10000 0
20000 0
40000 0
80000 0
160000 1
320000 2
640000 3
1280000 6
2560000 12
5120000 25
10240000 49
20480000 98
40960000 206
81920000 406

Fin de la medicion de tiempos *****
```

Los primeros tiempos son tan bajos porque los primeros números son muy pequeños para el algoritmo de suma, y ejecuta las operaciones muy rápido, dando resultados menores a 1ms.

Gráfica de complejidad usando –Xint Vector3



Ejecuciones usando -Xint Vector4

```
C:\Users\uo307928\Desktop\alg_NoelSuarezMenendezU0307928\p11>java -Xint p11.Vector4 1
repeticiones = 1
n      Tiempo
10000  0
20000  0
40000  0
80000  0
160000 1
320000 2
640000 3
1280000 6
2560000 12
5120000 25
10240000 49
20480000 97
40960000 198
81920000 399

Fin de la medicion de tiempos *****
```

```
C:\Users\uo307928\Desktop\alg_NoelSuarezMenendezU0307928\p11>java -Xint p11.Vector4 10
repeticiones = 10
n      Tiempo
10000  1
20000  1
40000  2
80000  3
160000 8
320000 15
640000 31
1280000 62
2560000 124
5120000 249
10240000 493
20480000 993
40960000 2012
81920000 3984

Fin de la medicion de tiempos *****
```

```
C:\Users\uo307928\Desktop\alg_NoelSuarezMenendezU0307928\p11>java -Xint p11.Vector4 100
repeticiones = 100
n      Tiempo
10000  4
20000  10
40000  19
80000  39
160000 77
320000 152
640000 308
1280000 621
2560000 1256
5120000 2544
10240000 5034
20480000 9875
40960000 19793
81920000 39659

Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4 1000
repeticiones = 1000
n      Tiempo
10000  53
20000  103
40000  222
80000  493
160000 989
320000 1954
640000 4426
1280000 12769
2560000 15471
5120000 26483
10240000 51801
20480000 101426
40960000 224756
81920000 490752

Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4 10000
repeticiones = 10000
n      Tiempo
10000  471
20000  933
40000  1877
80000  3763
160000 7424
320000 15031
640000 30764
1280000 61769
2560000 125580
5120000 274393
10240000 512944
20480000 1024004
40960000 2079285
81920000 4076783

Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4 100000
repeticiones = 100000
n      Tiempo
10000  4808
20000  9628
40000  19011
80000  38452
160000 75878
320000 156546
640000 317757
1280000 617601
2560000 1306901
5120000 2551571
10240000 5137788
20480000 10179078
```

No pude acabar la ejecución de Vector4 con 100.000 repeticiones porque tardo más de 10 horas y todavía no había llegado a $n = 40960000$.

SE LE PIDE: ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para $k=3$ y $k=4$ y compruebe los tiempos obtenidos.)

¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal $O(n)$?

A partir de lo visto en Vector4.java midiendo los tiempos de suma, realice las tres siguientes Clases: Vector5.java para medir los tiempos del máximo, Vector6.java para medir los tiempos de coincidencias1 y Vector7.java para medir los tiempos de coincidencias2.

Como el algoritmo suma tiene complejidad $O(n)$, si el tamaño del problema se multiplica por 2, el número de operaciones también se multiplica por 2. En consecuencia, aproximadamente el tiempo de ejecución se duplica.

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4 1
repeticiones = 1
n      Tiempo
10000  0
20000  0
40000  0
80000  0
160000 0
320000 2
640000 3
1280000 8
2560000 14
5120000 27
10240000 62
20480000 147
40960000 251
81920000 684
Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4k3 1
repeticiones = 1
n      Tiempo
10000  0
20000  0
40000  0
80000  0
160000 1
320000 2
640000 4
1280000 8
2560000 17
5120000 36
10240000 49
20480000 120
40960000 244
81920000 450
163840000 958
Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4k4 1
repeticiones = 1
n      Tiempo
10000  0
20000  0
40000  0
80000  0
160000 1
320000 3
640000 5
1280000 7
2560000 19
5120000 30
10240000 56
20480000 109
40960000 253
81920000 503
163840000 1010
327680000 1609
Fin de la medicion de tiempos *****
```

Sí, los tiempos son coherentes con la complejidad $O(n)$. Al multiplicar el tamaño del problema, el tiempo crece en esa proporción más o menos.


```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector4 300
repeticiones = 300
n      Tiempo
10000   16
20000   29
40000   56
80000  111
160000 229
320000 452
640000 905
1280000 1786
2560000 3733
5120000 7426
10240000 17592
20480000 77923
40960000 119731
81920000 138156

Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector5 300
repeticiones = 300
n      Tiempo
10000   28
20000   36
40000   75
80000  149
160000 320
320000 649
640000 1269
1280000 2609
2560000 5251
5120000 10422
10240000 20493
20480000 51077
40960000 87486
81920000 168532

Fin de la medicion de tiempos *****
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector6 1
repeticiones = 1
n      Tiempo
10000   613
20000  2632
40000 10183
80000 41681
160000 170764
320000 1207459
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p11.Vector7 1
repeticiones = 1
n      Tiempo
10000   0
20000   0
40000   0
80000   1
160000  3
320000  3
640000  4
1280000 10
2560000 24
5120000 42
10240000 84
20480000 158
40960000 355
81920000 687

Fin de la medicion de tiempos *****
```

P12

SE LE PIDE: Tras medir los tiempos, rellenar la tabla (tabla 1):

n	TBucle1	TBucle2	TBucle3	TBucle4
100	0	1	1	1
200	0	1	4	6
400	0	3	15	45
800	0	14	65	346
1600	1	56	270	2661
3200	2	259	1104	20521
6400	2	1053	4615	166164
12800	2	4180	19661	Fdt
25600	4	18292	82100	Fdt
51200	7	74966	345408	Fdt

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los cuatro casos.

Sí, ya que el que menos complejidad tiene, Bucle1 con $O(n \log n)$, es el que menos tarda, y los siguientes vas subiendo los tiempos de forma equivalente a sus complejidades hasta llegar al Bucle4, el cual tiene la complejidad más alta, $O(n^3)$.

SE LE PIDE: Implementar tres nuevas clases Bucle5, Bucle6 y Bucle7, que simulen algoritmos iterativos con una complejidad $O(n^2 \log n^2)$, $O(n^3 \log n)$ y $O(n^4)$ respectivamente. Tras implementar las clases, mida sus tiempos de ejecución y rellene la tabla (tabla 2):

Estos bucles parten de un $n = 5$, para poder realizar las medidas.

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p12.Bucle5 1
n      tiempo  repeticiones  contador
100     1        1          140000
200     5        1          640000
400    20        1         2880000
800    87        1        12800000
1600   382       1        56320000
3200  1677       1       245760000
6400  6991       1      1064960000
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p12.Bucle6 1
n      tiempo  repeticiones  contador
100     64        1          7000000
200    536        1         64000000
400   4962        1        576000000
800  43555        1       5120000000
1600 345328        1      45056000000
```

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928>java -Xint p12.Bucle7 1
n      tiempo  repeticiones  contador
100    30        1          6250000
200   457        1        100000000
400  7417        1       1600000000
800 149391        1      25600000000
```

n	TBucle5	TBucle6	TBucle7
100	1	64	30

200	5	536	457
400	20	4962	7417
800	87	43555	149391
1600	382	345328	Fdt
3200	1677	Fdt	Fdt
6400	6991	Fdt	Fdt

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los tres casos.

Concuerdan, ya que el que más complejidad tiene, que es Bucle 4, es el que mayores tiempos obtiene, mientras que el que menos complejidad tiene, que es Bucle5, es el que menores tiempos obtiene.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla (tabla 3):

n	TBucle1	TBucle2	t1/t2
100	0	1	0
200	0	1	0
400	0	3	0
800	0	14	0
1600	1	56	0,017857
3200	2	259	0,007722
6400	2	1053	0,001899
12800	2	4180	0,000478
25600	4	18292	0,000219
51200	7	74966	0,000093

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Sí, ya que al ser menor la complejidad de Bucle1 a la de Bucle2, estamos dividiendo tiempos muy pequeños (los de Bucle1) entre tiempos mucho mayores (los de Bucle2), por lo que el cociente es un valor muy pequeño.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla (tabla 4):

n	TBucle3	TBucle2	T3/t2
100	1	1	1
200	4	1	4
400	15	3	5
800	65	14	4,642857
1600	270	56	4,821429
3200	1104	259	4,262548
6400	4615	1053	4,382716
12800	19661	4180	4,703589
25600	82100	18292	4,488301
51200	345408	74966	4,607529

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Sí, ya que al dividir los tiempos de Bucle3 entre los de Bucle2, estamos dividiendo tiempos más grandes entre tiempos más pequeños (ya que la complejidad de Bucle3 es mayor que la de Bucle2), por lo que el cociente es un valor mayor que uno y fijo, ya que se conserva la proporción.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla (tabla 5):

n	Bucle4 Python	Bucle4 Java Sin Optimizar	tPython/tJava
200	29	6	4,83
400	198	45	4,4
800	1710	346	4,92
1600	14929	2661	5,61
3200	Fdt	20521	No medible
6400	Fdt	6	No medible

Razone si los diferentes tiempos y sus cocientes concuerda con lo esperado.

Los valores siguen un patrón según aumentan las iteraciones, pero entre el algoritmo ejecutado en Java sin optimizar y el de Python, vemos que el de Python tiene tiempos mucho mayores.

P2

SE LE PIDE: Tras medir los tiempos, rellenar la tabla (tabla 1):

Parámetro usado: 1

n	t ordenado	t inverso	t aleatorio
10000	432	1116	880
20000	1676	4357	3479
40000	6909	17529	13793
80000	27113	69867	55345
160000	107978	278916	221901
320000	Fdt	Fdt	Fdt
640000	Fdt	Fdt	Fdt
1280000	Fdt	Fdt	Fdt

El algoritmo Burbuja no varía su complejidad en función de si sus elementos están ordenados o no, siempre es $O(n^2)$.

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Los tiempos obtenidos concuerdan con lo esperado según la complejidad teórica del algoritmo burbuja. Al duplicar el tamaño del problema el tiempo se multiplica aproximadamente por cuatro, lo que indica un comportamiento cuadrático.

SE LE PIDE: Implementar una clase SeleccionTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla (tabla 2):

Parámetro usado: 1

n	t ordenado	t inverso	t aleatorio
10000	394	349	387
20000	1585	1401	1579
40000	6355	5616	6320
80000	25247	22053	25402
160000	101409	88122	100765
320000	Fdt	Fdt	Fdt
640000	Fdt	Fdt	Fdt
1280000	Fdt	Fdt	Fdt

El algoritmo de Selección es como el Burbuja, no varía su complejidad en función de si sus elementos están ordenados o no y siempre es $O(n^2)$.

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Los tiempos obtenidos concuerdan con la complejidad teórica del algoritmo Selección. Al duplicar el tamaño del problema el tiempo se multiplica aproximadamente por cuatro, lo que indica un comportamiento cuadrático

SE LE PIDE: Implementar una clase InsercionTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla (tabla 3):

Parámetro usado: 1

n	t ordenado	t inverso	t aleatorio
10000	1	675	337
20000	1	2719	1381
40000	1	10898	5433
80000	2	43544	21817
160000	3	173786	87664
320000	6	Fdt	Fdt
640000	13	Fdt	Fdt
1280000	24	Fdt	Fdt

La complejidad del algoritmo de inserción varía si sus elementos están ordenados o no. El mejor caso es cuando el vector esta ordenado, ya que cada elemento se compara solo una vez con el anterior, dando una complejidad $O(n)$, mientras que, si está desordenado o inversamente ordenado, la complejidad es $O(n^2)$, ya que cuando esta desordenado cada elemento se compara con la mitad de los elementos anteriores, y cuando esta inversamente ordenado se compara con todos los elementos anteriores, siendo este el peor caso.

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Los resultados concuerdan perfectamente con la complejidad teórica, ya que en el caso en el que el vector está ordenado, el crecimiento es lineal, y en los demás casos el crecimiento es cuadrático, teniendo tiempos más grandes en el peor caso, el inversamente ordenado.

SE LE PIDE: Implementar una clase RapidoTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla (tabla 4):

Parámetro usado: 10

n	t ordenado	t inverso	t aleatorio
10000	25	31	36
20000	53	58	77
40000	110	121	167
80000	229	258	353
160000	494	545	760
320000	1058	1142	1625
640000	2165	2386	3335
1280000	4501	4980	7077

Este algoritmo obtiene una complejidad quasi-lineal en los tres casos, cuando esta ordenado, aleatorio o inversamente ordenado, teniendo casi siempre una complejidad de $O(n \log n)$, ya que elige el pivote central y evita particiones degeneradas. El único caso en el que la complejidad puede ser peor es cuando el pivote es el elemento más pequeño o el más grande, dando una complejidad $O(n^2)$.

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Los resultados experimentales concuerdan con la complejidad teórica del algoritmo implementado, ya que el tiempo crece aproximadamente por un factor cercano a 2 al duplicar n (comportamiento de la complejidad $O(n \log n)$), presentando en los tres casos tiempos similares.

Proyete, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 861400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de $n= 811920.000$ (o sea, $n=80000 \cdot 24 \cdot 26=80000 \cdot 210$), en el caso inicialmente en orden aleatorio?

Tomando como referencia $n_0 = 160\,000$ (último valor antes de Fdt en varios algoritmos), nuestro n ahora es 811920000, por lo que aumentamos n en un factor de 5074.5.

Para el algoritmo Burbuja, al ser cuadrático, como $t_0 = 22190\text{ms}$, $t = t_0 \cdot k^2$, con $k^2 = 25750000$, por lo que $t = 5,71 \times 10^{12}$ ms o aproximadamente 66300 días.

Para el algoritmo de selección el cálculo es el mismo, solo que ahora $t_0 = 100765$ ms, por lo que $t = 2,59 \times 10^{12}$ ms o aproximadamente 30100 días.

Para el algoritmo de inserción en el caso de un vector aleatorio, la complejidad es la

misma, por lo que el cálculo es el mismo pero con $t_0 = 87664$ ms, por lo que $t = 2,26 \times 10^{12}$ ms o aproximadamente 26200 días.

Por último, para el algoritmo Rápido, con complejidad $O(n \log n)$, y con $t_0 = 760$ ms,

$t = t_0 \cdot k \cdot \log n / \log n_0$ por lo que $t = 760 \cdot 5074.5 \cdot 1.42 = 5,48 \times 10^6$ ms o 0,064 días.

SE LE PIDE: Implementar dos clases (RapidoTiemposBajos.java e InsercionTiemposBajos.java), que tras ser ejecutadas sirvan para rellenar la siguiente tabla:

n	t rápido	t inserción	t rápido/t inserción
10	109	15	7,27
15	194	20	9,70
20	283	36	7,86
25	332	48	6,92
30	450	66	6,82
35	529	95	5,57
40	626	122	5,13
45	769	160	4,81
50	832	192	4,33
55	935	242	3,86
60	1039	274	3,79
65	1155	326	3,54
70	1266	386	3,28
75	1385	418	3,31
80	1511	469	3,22
85	1570	532	2,95
90	1681	597	2,82
95	1848	648	2,85
100	1959	711	2,76

Razone a partir de qué valor de n comienza a tardar menos tiempo el algoritmo de menor complejidad (rápido) que el de mayor complejidad (inserción).

El algoritmo rápido comenzará a tardar menos tiempo que el de inserción para valores de n suficientemente grandes. En la tabla la relación de tiempos disminuye progresivamente, lo que concuerda con que rápido tiene complejidad $O(n \log n)$ frente a $O(n^2)$ de inserción. Como para un $n = 10$, la relación de rápido e inserción es 7,27 y para un $n = 100$ la relación es 2,76 y calculamos como avanza la ejecución, por lo que el algoritmo rápido tendrá tiempos menores que el de inserción para un $n = 280$, ya que habría que multiplicar el tiempo del algoritmo rápido por 2,8 más o menos, lo que darían 5485 ms, y el tiempo de inserción lo habría que multiplicar por $2,8^2$ (ya que es cuadrático), lo que darían unos 5574 ms.

A partir del análisis de los resultados obtenidos en el apartado anterior, puede diseñar e implementar un algoritmo que combine el método rápido con el método

inserción, teniendo como objetivo bajar los tiempos obtenidos por el rápido y reflejados en la TABLA4.

Calcular los tiempos del algoritmo diseñado y compararlos con los del rápido, para acabar concluyendo en qué grado se consiguió el objetivo antes enunciado.

P3

SE LE PIDE: Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Para sustracción 1, la complejidad del método es $O(1)$, teniendo $a=1$, $b=1$ y $k=0$, por lo que la complejidad temporal es $O(n^{k+1})$ que es $O(n^{0+1})$, $O(n)$.

Para sustracción 2, la complejidad del método es $O(n)$ al haber un bucle for, teniendo $a=1$, $b=1$ y $k=1$, por lo que la complejidad temporal es $O(n^{k+1})$ que es $O(n^{1+1})$, $O(n^2)$.

Para sustracción 3, $a=2$ (ya que hay 2 llamadas recursivas) y $b=1$, por lo que la complejidad temporal es $O(a^{n/b})$ que es $O(2^n)$.

Contestar: ¿para qué valor de n dejan de dar tiempo (abortan) las clases Sustracción1 y Sustracción2? ¿por qué sucede eso?

Para un $n = 8000$, ya que se produce un desbordamiento de la pila debido a la cantidad de llamadas recursivas.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción3 para $n=80$? Razonar la respuesta.

De ejecutarlo 1 vez se saca que para $n = 32$, $t=180086$ ms ≈ 180 s, por lo que $T(80)=T(32) \cdot 2^{80-32} = 180 \cdot 2^{48} = 180 \cdot 2,56 \cdot 10^{14} = 4.608 \cdot 10^{16}$ segundos = $1,46 \cdot 10^9$ años

Implementar una clase Sustracción4.java con una complejidad $O(n^3)$ (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para $n=100, 200, 400, 800, \dots$ (hasta FdT).

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++){
            for (int j=0;j<n;j++){
                cont ++;
            }
        }
    }
}
```



```

    }
    }
    rec4 (n-1);
  }
}

```

n	t Sustracción4
10	0
20	0
40	0
80	3
160	9
320	72
640	576
1280	4597
2560	36978

Al duplicar n, el tiempo se multiplica por 8.

Implementar una clase Sustracción5.java con una complejidad $O(3^{n/2})$ (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, ... (hasta FdT).

```
public static void rec5 (int n)
```

```

{
    if (n<=0)
        cont++;
    else
    {
        cont++; // O(1)
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);
    }
}

```

n	t Sustracción5
30	400
32	1190
34	3569
36	10717

38	32117
40	96519
42	Fdt

Al incrementar n 2 unidades, el tiempo se multiplica por 3.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80?

Razonar la respuesta.

Calculando la relación entre el último n del que se tienen tiempos, que es n = 40, y n = 80, tenemos que $T(80)/T(40) = 3^{80/2}/3^{40/2} = 3^{40}/3^{20} = 3^{20} = 3,49 \times 10^9$, por lo que $T(80) = 96.519 \cdot 3,49 \times 10^9$ y para pasarlo a años hay que dividir entre $3,15 \times 10^7$ ms que tiene un año, lo que da $1,07 \times 10^4$ años.

SE LE PIDE: Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Para división 1, la complejidad del método es $O(n/3)$, teniendo a=1, b=3 y k=1, por lo que la complejidad temporal es $O(n^k)$ que es $O(n^1)$, $O(n)$.

Para división 2, la complejidad del método es $O(n)$ al haber un bucle for (k=1), teniendo a=2 y b=2, por lo que la complejidad temporal es $O(n^k \cdot \log n)$ que es $O(n^1 \cdot \log n)$, $O(n \log n)$.

Para división 3 la complejidad es $O(1)$, por lo que a=2, b=2 y k=0, como $a > b^k$, la complejidad temporal es $O(n^{\log_b a})$ que es $O(n^{\log_2 2})$, $O(n)$.

Implementar una clase Division4.java con una complejidad $O(n^2)$ (con $a < b^k$) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=1000, 2000, 4000, 8000, ... (hasta FdT).

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=1;i<n;i++)
        {
            for (int i=1;i<n;i++)
            {
                count++; // k = 2
            }
        }
        rec1 (n/3); // b = 3 y a = 1
    }
}
```

```
    }
}
```

Al duplicar n , el tiempo se multiplica por 4.

Implementar una clase Division5.java con una complejidad $O(n^2)$ (con $a > b^k$) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para $n=1000, 2000, 4000, 8000, \dots$ (hasta FdT).

```
public static void rec5 (int n)
```

```
{
    if (n<=0)
        cont++;
    else
    {
        cont++ ; // O(1)
        rec3 (n/2);
        rec3 (n/2);
        rec3 (n/2);
        rec3 (n/2);
    }
}
```

Al duplicar n , el tiempo se multiplica por 4 (un poco mas)

SE LE PIDE: Tras analizar la complejidad de los diversos algoritmos que hay dentro de las dos clases, ejecutarlas y tras poner en una tabla los tiempos obtenidos, comparar la eficiencia de cada algoritmo.

Ambos ejecutados con parámetro 5.

n	Division4	Division5
10000	37	123
20000	154	497
40000	587	2070
80000	2377	15317
160000	24470	48609

Los tiempos de Division4 dan un crecimiento de un factor por 4 al duplicar n , lo que concuerda con la complejidad cuadrática $O(n^2)$. En cambio, Division5 presenta tiempos significativamente mayores pese a que tienen la misma complejidad, lo que indica menor eficiencia.

SE LE PIDE:

PRIMERA PARTE

Resolver el problema mediante un algoritmo trivial con una clase

p3p.PuntosTrivial.java, que calcule la solución calculando la distancia entre todo par de puntos diferentes, para así saber al final el par de puntos buscado y su distancia mínima.

Ejecutar la clase metiendo como parámetro cada uno de los nombres de los ficheros aportados, para comprobar el buen funcionamiento del algoritmo implementado:

`java p3p.PuntosTrivial nombreFichero // [datos32.txt .. datos8192.txt]`

Hacer una clase **p3p.PuntosTrivialTiempos.java** que tras generar de forma aleatoria los *n* puntos, posteriormente calcule el tiempo de ejecución del algoritmo trivial:

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosTrivial p3p/datos8192.txt
PUNTOS MÁS CERCANOS: [83,681527, 10,289108] [83,691485, 10,288760]
SU DISTANCIA MÍNIMA= 0,009964

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosTrivial p3p/datos2048.txt
PUNTOS MÁS CERCANOS: [27,692774, 55,783120] [27,686908, 55,818825]
SU DISTANCIA MÍNIMA= 0,036184

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosTrivial p3p/datos512.txt
PUNTOS MÁS CERCANOS: [68,474715, 94,638993] [68,511846, 94,653169]
SU DISTANCIA MÍNIMA= 0,039745

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosTrivial p3p/datos128.txt
PUNTOS MÁS CERCANOS: [91,434401, 16,793360] [91,160139, 17,465655]
SU DISTANCIA MÍNIMA= 0,726086

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosTrivial p3p/datos32.txt
PUNTOS MÁS CERCANOS: [27,278446, 90,131051] [27,536642, 89,236468]
SU DISTANCIA MÍNIMA= 0,931098
```

n	t
datos32.txt	0
datos128.txt	2
datos512.txt	13
datos2048.txt	34
datos8192.txt	87

Razonar si sintonizan los tiempos obtenidos con la complejidad del algoritmo.

El algoritmo es cuadrático $O(n^2)$, ya que calcula la distancia entre cada par de puntos y el resto. En teoría, al duplicar el tamaño del problema el tiempo debería multiplicarse aproximadamente por cuatro. En los resultados obtenidos se observa que al aumentar el tamaño de la entrada el tiempo de ejecución también crece de forma significativa, lo cual es coherente con un crecimiento cuadrático. Aunque el incremento no es de 16 al multiplicar *n* por 4, se debe a que para tamaños pequeños influye la resolución de `System.currentTimeMillis()`.

SEGUNDA PARTE

Resolver el problema mediante un algoritmo Divide y Vencerás con una clase

p3p.PuntosDyV.java que permita, dividiendo el problema en subproblemas, saber al final el par de puntos buscado y su distancia mínima.

Ejecutar la clase metiendo como parámetro cada uno de los nombres de los ficheros aportados para comprobar el buen funcionamiento del algoritmo implementado:

java p3p.PuntosDyV nombreFichero // datos32.txt, ..., datos2048.txt

Hacer una clase p3p.PuntosDyVTiempos.java que tras generar de forma aleatoria los n puntos, posteriormente calcule el tiempo de ejecución del algoritmo DyV:

```
C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosDyV p3p/datos32.txt
PUNTOS MÁS CERCANOS: [24,165626, 89,108510] [27,278446, 90,131051]
DISTANCIA MÍNIMA = 3,276467

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosDyV p3p/datos128.txt
PUNTOS MÁS CERCANOS: [28,992827, 35,190279] [29,801051, 35,031723]
DISTANCIA MÍNIMA = 0,823630

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosDyV p3p/datos512.txt
PUNTOS MÁS CERCANOS: [99,411413, 72,662098] [99,538711, 72,638045]
DISTANCIA MÍNIMA = 0,129550

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosDyV p3p/datos2048.txt
PUNTOS MÁS CERCANOS: [79,176821, 61,738456] [79,185432, 61,665744]
DISTANCIA MÍNIMA = 0,073220

C:\Users\Usuario\Desktop\alg_NoelSuarezMenendezU0307928\p3>java -cp . p3p.PuntosDyV p3p/datos8192.txt
PUNTOS MÁS CERCANOS: [83,681527, 10,289108] [83,691485, 10,288760]
DISTANCIA MÍNIMA = 0,009964
```

n	t
datos32.txt	1
datos128.txt	1
datos512.txt	1
datos2048.txt	1
datos8192.txt	3

Razone si sintonizan los tiempos obtenidos con la complejidad del algoritmo.

En los tiempos obtenidos se ve que el tiempo casi no aumenta al multiplicar el tamaño de n desde 32 hasta 8192 puntos, lo que se debe a que el algoritmo es muy eficiente y n todavía tiene valores muy pequeños para que se note el crecimiento. Además, al utilizar System.currentTimeMillis(), cuya resolución es de milisegundos, las diferencias de tiempo para ejecuciones rápidas no se aprecian con tanta precisión. Aun así, los tiempos son más o menos coherentes con la complejidad $O(n \log n)$, ya que el tiempo crece muy lentamente en comparación con el algoritmo trivial $O(n^2)$.

Opcional, hacer una clase p3p.PuntosTodosDyV.java que resuelva el problema planteado para cualquier número de puntos $n \geq 3$.

Pruebe el buen funcionamiento de p3p.PuntosTodosDyV.java para: “datos100.txt” : para este caso la solución ha de ser: PUNTOS MÁS CERCANOS: [40.203173, 17.809812] [39.518531, 18.864816] SU DISTANCIA MÍNIMA= 1.257684

“datos1000.txt” : para este caso la solución ha de ser: PUNTOS MÁS CERCANOS: [8.036015, 65.015139] [8.005602, 64.992083] SU DISTANCIA MÍNIMA= 0.038165

“datos10000.txt” : para este caso la solución ha de ser: PUNTOS MÁS CERCANOS: [73.085395, 31.480901] [73.085113, 31.479733] SU DISTANCIA MÍNIMA= 0.001202

P4

SE LE PIDE: Implementar una clase `ColoreoGrafo.java`, de modo que con la clase proporcionada `Devorador.java` podamos calcular una solución y visualizarla con el módulo proporcionado en Python. Se debe añadir el JAR provisto al build path para que este último funcione. Razone convenientemente la complejidad temporal del algoritmo implementado.

El algoritmo recorre todos los nodos del grafo y para cada nodo se recorren sus vecinos utilizando la lista de adyacencia del map.

Si n es el número de nodos y A el número de aristas, recorrer todos los nodos tiene coste $O(n)$, mientras que recorrer los vecinos de cada nodo equivale a recorrer las aristas del nodo (de cada arista sale un nodo vecino), por tanto, la complejidad total del algoritmo es $O(n+A)$

En grafos dispersos, es decir, con pocas aristas (donde A se aproxima a n) el coste es aproximadamente lineal, mientras que en grafos densos (donde A es más o menos n^2) el coste puede aproximarse a $O(n^2)$.

Implementar una clase `DevoradorTiempos.java`, que emplee los grafos contenidos en `sols` y calculando el tiempo que tarda el algoritmo hecho en el apartado anterior en resolver el problema, de forma que se pueda ir rellenando la siguiente tabla.

Razone: ¿siguen la complejidad calculada los tiempos de la tabla?

n	t Coloreado (ms)
4	6
8	2
16	9
32	11
64	25
100	70
128	53
256	84
512	78
1024	202
2048	428
4096	940
8192	1875
16384	5260
32768	11027

65536	30102
-------	-------

Con este algoritmo los tiempos escalan de una manera muy lineal al aumentar la n (si la duplicamos vemos que el tiempo se multiplica por 2).

P5

Se le pide:

1. Definición del estado: explicar claramente qué significan los índices de tu tabla/estructura de datos.

Se define una tabla de programación dinámica $DP[i][l]$, donde i representa el número de vehículo, los i primeros, y l representa la longitud total ocupada en el carril de babor.

Si $DP[i][l]$ es true significa que es posible meter los i primeros vehículos, ocupando l metros en babor. La ocupación de estribor no hace falta ponerla porque se deduce partir de S_i y los metros ocupados en babor

$$\text{estribor} = S[i] - l$$

donde $S[i]$ es la suma acumulada de las longitudes de los i primeros vehículos.

2. Relación de recurrencia: escribir la formulación matemática que permite calcular el estado (i, j) a partir de los estados anteriores. Debe incluir los casos base.

Para cada vehículo i de longitud v_i , $DP[i][l]$ se puede sacar a partir de estados anteriores, ya que i se coloca en babor si $DP[i-1][l-v_i] = \text{true}$ para $l-v_i \geq 0$. Esto es porque, si previamente era posible ocupar $l-v_i$ metros con $i-1$ vehículos, entonces ahora es posible ocupar l añadiendo el vehículo i en babor. Se coloca en estribor si $DP[i-1][l]$ si $S[i]-l \leq L$, donde la ocupación de babor no cambia, pero se comprueba que la ocupación de estribor no supera la capacidad del ferry. El caso base (estando toda la matriz inicializada a false) $DP[0][0] = \text{true}$ significa que 0 vehículos caben en 0 metros de Ferry.

3. Complejidad: análisis del coste temporal del algoritmo en función de N (número de vehículos) y L (longitud del ferry).

Como para cada vehículo se comprueba si cabe en cada longitud, la complejidad del algoritmo sería $O(N \cdot L)$.

4. Implementación: código en la clase `p5.Ferry.java` que resuelva el problema aceptando el fichero de entrada como parámetro.

P6

Se le pide:

Una clase `p6.AlmacenajeContenedores.java` y una clase `p6.AlmacenajeContenedoresTiempos.java`.

Rellene una tabla en la que aparezcan para los ficheros antes indicados los tiempos obtenidos (en milisegundos).

Tabla de la ejecución del algoritmo `AlmacenajeContenedoresTiempos` sin poda y sin optimización.

fichero	Tiempo (ms)	Contenedores	Llamadas R
<u>test00.txt</u>	5	4	136
<u>test01.txt</u>	5	5	536
<u>test02.txt</u>	806	7	1036339
<u>test03.txt</u>	16	6	19554
<u>test04.txt</u>	273433	10	222281543
<u>test05.txt</u>	Fdt	-	-
<u>test06.txt</u>	46	7	55190
<u>test07.txt</u>	66	8	60108
<u>test08.txt</u>	Fdt	-	-
<u>test09.txt</u>	Fdt	-	-

Tabla de la ejecución del algoritmo `AlmacenajeContenedoresTiempos` sin poda y con optimización.

fichero	Tiempo (ms)	Contenedores	Llamadas R
<u>test00.txt</u>	5	4	136
<u>test01.txt</u>	3	5	536
<u>test02.txt</u>	50	7	1036339
<u>test03.txt</u>	2	6	19554
<u>test04.txt</u>	2986	10	222281543
<u>test05.txt</u>	482311	14	1700689983
<u>test06.txt</u>	3	7	55190
<u>test07.txt</u>	2	8	60108
<u>test08.txt</u>	Fdt	-	-
<u>test09.txt</u>	Fdt	-	-

Tabla de la ejecución del algoritmo AlmacenajeContenedoresTiempos con poda y con optimización.

fichero	Tiempo (ms)	Contenedores	Llamadas R
<u>test00.txt</u>	4	4	8
<u>test01.txt</u>	1	5	12
<u>test02.txt</u>	3	7	20
<u>test03.txt</u>	3	6	15
<u>test04.txt</u>	1	10	34
<u>test05.txt</u>	10	14	31
<u>test06.txt</u>	5	7	16
<u>test07.txt</u>	1	8	17
<u>test08.txt</u>	128	19	326611
<u>test09.txt</u>	Fdt	-	-

Lista de contenedores y objetos contenidos para cada test:

Contenedor 1: 7 3
 Contenedor 2: 6 4
 Contenedor 3: 5 5
 Contenedor 4: 2

Contenedor 1: 11 1
 Contenedor 2: 10 2
 Contenedor 3: 9 3
 Contenedor 4: 8 4
 Contenedor 5: 4

Contenedor 1: 9 1
 Contenedor 2: 8 2
 Contenedor 3: 7 3
 Contenedor 4: 6 4
 Contenedor 5: 6 4
 Contenedor 6: 5 5
 Contenedor 7: 4 3 2

Contenedor 1: 11 1
 Contenedor 2: 10 2
 Contenedor 3: 9 3

Noel Suárez Menéndez

Contenedor 4: 8 4

Contenedor 5: 6 6

Contenedor 6: 5 2

Contenedor 1: 14 1

Contenedor 2: 13 2

Contenedor 3: 12 3

Contenedor 4: 11 4

Contenedor 5: 10 5

Contenedor 6: 9 6

Contenedor 7: 9 6

Contenedor 8: 8 7

Contenedor 9: 8 7

Contenedor 10: 5 3

Contenedor 1: 19 1

Contenedor 2: 18 2

Contenedor 3: 17 3

Contenedor 4: 17

Contenedor 5: 16 4

Contenedor 6: 15 5

Contenedor 7: 14 6

Contenedor 8: 14 5

Contenedor 9: 13 7

Contenedor 10: 12 8

Contenedor 11: 12

Contenedor 12: 11 9

Contenedor 13: 11 9

Contenedor 14: 10 10

Contenedor 1: 13 1

Contenedor 2: 12 2

Contenedor 3: 11 3

Contenedor 4: 10 4

Contenedor 5: 9 5

Contenedor 6: 8 6

Contenedor 7: 7 7

Contenedor 1: 14

Contenedor 2: 13 2

Contenedor 3: 12 3

Contenedor 4: 11 4

Contenedor 5: 10 5

Contenedor 6: 9 6

Noel Suárez Menéndez

Contenedor 7: 8 7

Contenedor 8: 8 6

Contenedor 1: 19 1

Contenedor 2: 18 2

Contenedor 3: 17 3

Contenedor 4: 17 3

Contenedor 5: 16 4

Contenedor 6: 16 4

Contenedor 7: 15 5

Contenedor 8: 15 5

Contenedor 9: 14 6

Contenedor 10: 14 6

Contenedor 11: 13 7

Contenedor 12: 13 7

Contenedor 13: 12 8

Contenedor 14: 12 8

Contenedor 15: 11 9

Contenedor 16: 11 9

Contenedor 17: 10 10

Contenedor 18: 7 6 5 2

Contenedor 19: 4 3 1

¿Qué complejidad asigna al algoritmo que calcula el número mínimo de contenedores a utilizar? ¿Por qué?

Voy a analizar la complejidad del algoritmo sin tener en cuenta la poda.

En este algoritmo n (el número de objetos a almacenar) es `conjuntoS.length`, y k es el número de contenedores, siendo n como máximo.

La altura del árbol (h) es n , siendo el caso base cuando se procesaron todos los objetos (`indexObject == conjuntoS.length`). Para el grado, si el objeto se mete en un contenedor existente hay k opciones, o si se mete en uno nuevo el número de ramas puede ser hasta $k+1$ (como máximo n). Como en cada llamada se recorren todos los contenedores, y en cada contenedor se suman sus elementos, eso tiene una complejidad $O(n^2)$, y como la complejidad del back tracking es el grado elevado a la altura, la complejidad del algoritmo es $O(n^n \cdot n^2)$, lo que se simplifica en $O(n^n)$.

P6-Extra

La clase implementa el problema de las N-Reinas mediante un algoritmo de backtracking optimizado, en el que se evita comprobar todo el tablero en cada paso.

En cada nivel del árbol se coloca una reina en una fila distinta.

La complejidad teórica de este algoritmo es $O(n!)$ pero se ve reducida por la poda.

Tiempos de ejecución del algoritmo de las reinas con y sin optimización:

```
C:\Users\uo307928\Desktop\alg_NoelSuarezMenendezU0307928\P6-Extra>java -Xint NReinasTiempos
Tamaño del tablero: 4    Tiempo: 0
Tamaño del tablero: 6    Tiempo: 0
Tamaño del tablero: 8    Tiempo: 2
Tamaño del tablero: 10   Tiempo: 11
Tamaño del tablero: 12   Tiempo: 306
Tamaño del tablero: 14   Tiempo: 10851

C:\Users\uo307928\Desktop\alg_NoelSuarezMenendezU0307928\P6-Extra>java NReinasTiempos
Tamaño del tablero: 4    Tiempo: 0
Tamaño del tablero: 6    Tiempo: 0
Tamaño del tablero: 8    Tiempo: 0
Tamaño del tablero: 10   Tiempo: 5
Tamaño del tablero: 12   Tiempo: 82
Tamaño del tablero: 14   Tiempo: 2213
```

Los resultados al ejecutar muestran un crecimiento exponencial de los tiempos, coherente con la complejidad teórica del problema de las N-Reinas mediante backtracking.

P7**Tabla de ejecuciones para comparar con el algoritmo sin poda**

fichero	Tiempo (ms)	Contenedores	Llamadas R
<u>test00.txt</u>	3	5	3
<u>test01.txt</u>	2	5	10
<u>test02.txt</u>	6	7	16
<u>test03.txt</u>	2	6	13
<u>test04.txt</u>	1	10	21
<u>test05.txt</u>	5	14	27
<u>test06.txt</u>	1	7	15
<u>test07.txt</u>	2	8	16
<u>test08.txt</u>	9	19	42
<u>test09.txt</u>	12	33	67

Resultado de ejecutar AlmacenajeContenedoresRyP con el test09.txt

Lista de contenedores y objetos contenidos:

Contenedor 1: 19 1
Contenedor 2: 18 2
Contenedor 3: 18 2
Contenedor 4: 17 3
Contenedor 5: 17 3
Contenedor 6: 17 3
Contenedor 7: 16 4
Contenedor 8: 16 4
Contenedor 9: 16 4
Contenedor 10: 16 4
Contenedor 11: 15 5
Contenedor 12: 15 5
Contenedor 13: 15 5
Contenedor 14: 15 5
Contenedor 15: 14 6
Contenedor 16: 14 6
Contenedor 17: 14 6
Contenedor 18: 14 6
Contenedor 19: 13 7
Contenedor 20: 13 7
Contenedor 21: 13 7
Contenedor 22: 13 7

Contenedor 23: 12 8
Contenedor 24: 12 8
Contenedor 25: 12 8
Contenedor 26: 12 8
Contenedor 27: 11 9
Contenedor 28: 11 9
Contenedor 29: 11 9
Contenedor 30: 11 9
Contenedor 31: 10 10
Contenedor 32: 10 10
Contenedor 33: 3 1

Conclusión

Hacer la poda de esta manera permite poder calcular los contenedores necesarios para el test09.txt, ya que se podan muchas posibles ramas, y como consecuencia reduce mucho el número de llamadas. El aspecto negativo es que el algoritmo para calcular el mejor, al ser un algoritmo hecho en clase, puede ser improductivo y hacer que el número de contenedores en una solución muy sencilla sea mayor, como pasa con el caso más básico que es el test00.txt.